



프로그램 언어

5주차



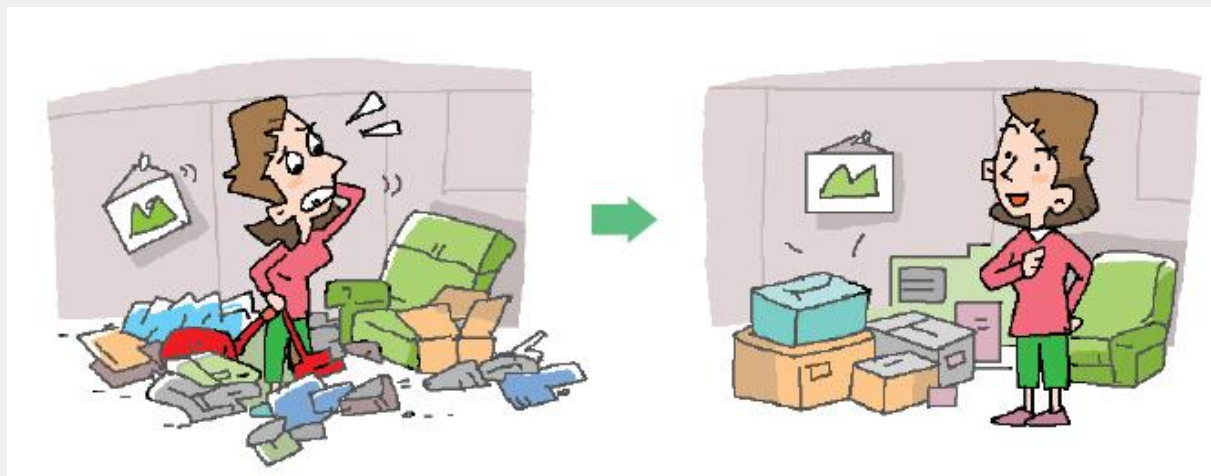
인하대학교



함수(Function)

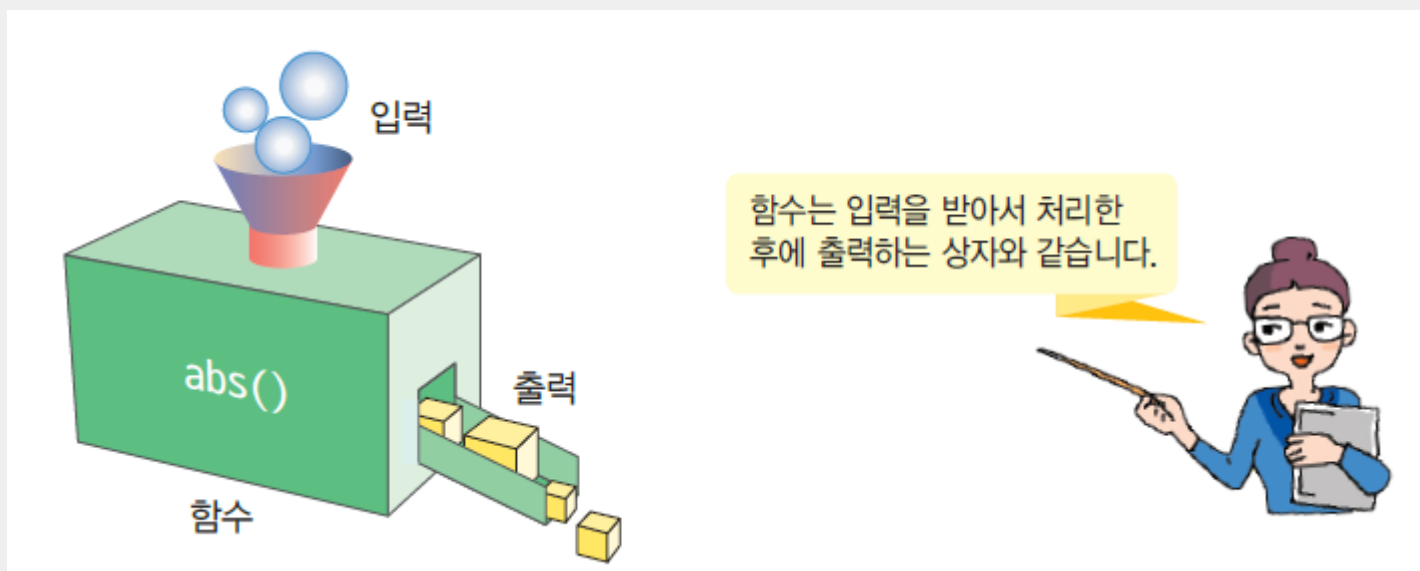
코드를 묶는 방법

- 관련 있는 코드들을 묶어서 전체 프로그램을 조직화할 필요가 있다.
- 코드를 묶는 3가지 방법
 - 함수(function)는 우리가 반복적으로 사용하는 코드를 묶은 것으로 프로그램의 빌딩 블록과 같다.
 - 객체(object)는 서로 관련 있는 변수와 함수를 묶는 방법이다.
 - 모듈(module)은 함수나 객체들을 소스 파일 안에 모은 것이다



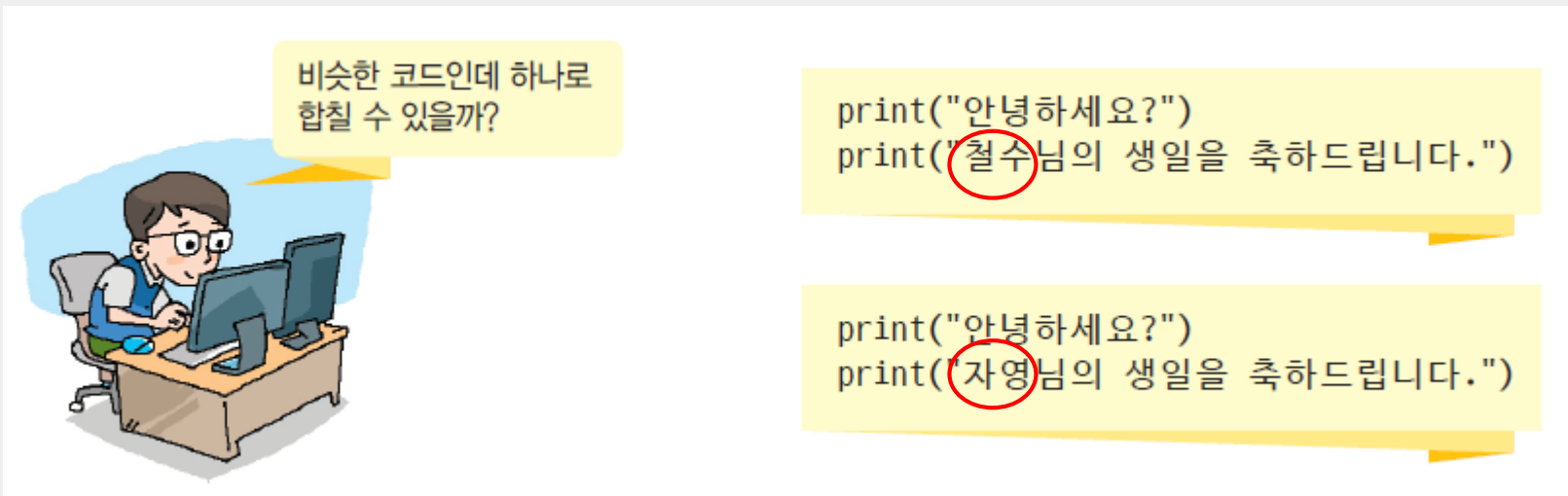
함수

- 함수(function)는 특정 작업을 수행하는 명령어들의 모음에 이름을 붙인 것이다.
- 함수는 작업에 필요한 데이터를 전달받을 수 있으며, 작업이 완료된 후에는 작업의 결과를 호출자에게 반환할 수 있다.



함수의 필요성

- 프로그램을 작성하다 보면 동일한 처리를 반복해야 하는 경우가 많이 발생한다.
- 여러 번 반복해야 되는 처리 단계를 하나로 모아서 필요할 때 언제든지 호출하여 사용할 수 있다



비슷한 코드인데 하나로
합칠 수 있을까?

```
print("안녕하세요?")  
print("철수님의 생일을 축하드립니다.")
```

```
print("안녕하세요?")  
print("자영님의 생일을 축하드립니다.")
```

내장함수



인하대학교

Python 3.x Resources

- Browse Python 3.11.5 Documentation - (Module Index)
 - What's new in Python 3.11
 - Tutorial
 - **Library Reference**
 - Language Reference
 - Extending and Embedding
 - Python/C API
 - Using Python
 - Python HOWTOs
 - Glossary

- Introduction
 - Notes on availability
- **Built-in Functions**
- Built-in Constants
 - Constants added by the `site` module
- Built-in Types
 - Truth Value Testing
 - Boolean Operations — `and`, `or`, `not`
 - Comparisons
 - Numeric Types — `int`, `float`, `complex`
 - Iterator Types
 - Sequence Types — `list`, `tuple`, `range`
 - Text Sequence Type — `str`
 - Binary Sequence Types — `bytes`, `bytearray`
 - Set Types — `set`, `frozenset`
 - Mapping Types — `dict`

Built-in Functions

A

`abs()`
`aiter()`
`all()`
`anext()`
`any()`
`ascii()`

B

`bin()`
`bool()`
`breakpoint()`
`bytearray()`
`bytes()`

C

`callable()`
`chr()`
`classmethod()`
`compile()`
`complex()`

D

`delattr()`
`dict()`
`dir()`
`divmod()`

E

`enumerate()`
`eval()`
`exec()`

F

`filter()`
`float()`
`format()`
`frozenset()`

G

`getattr()`
`globals()`

H

`hasattr()`
`hash()`
`help()`
`hex()`

I

`id()`
`input()`
`int()`
`isinstance()`
`issubclass()`
`iter()`

L

`len()`
`list()`
`locals()`

M

`map()`
`max()`
`memoryview()`
`min()`

N

`next()`

O

`object()`
`oct()`
`open()`
`ord()`

P

`pow()`
`print()`
`property()`

R

`range()`
`repr()`
`reversed()`
`round()`

S

`set()`
`setattr()`
`slice()`
`sorted()`
`staticmethod()`
`str()`
`sum()`
`super()`

T

`tuple()`
`type()`

V

`vars()`

Z

`zip()`

-

`__import__()`



사용자 정의 함수

```
def 함수이름(매개변수):
```

```
    실행문1
```

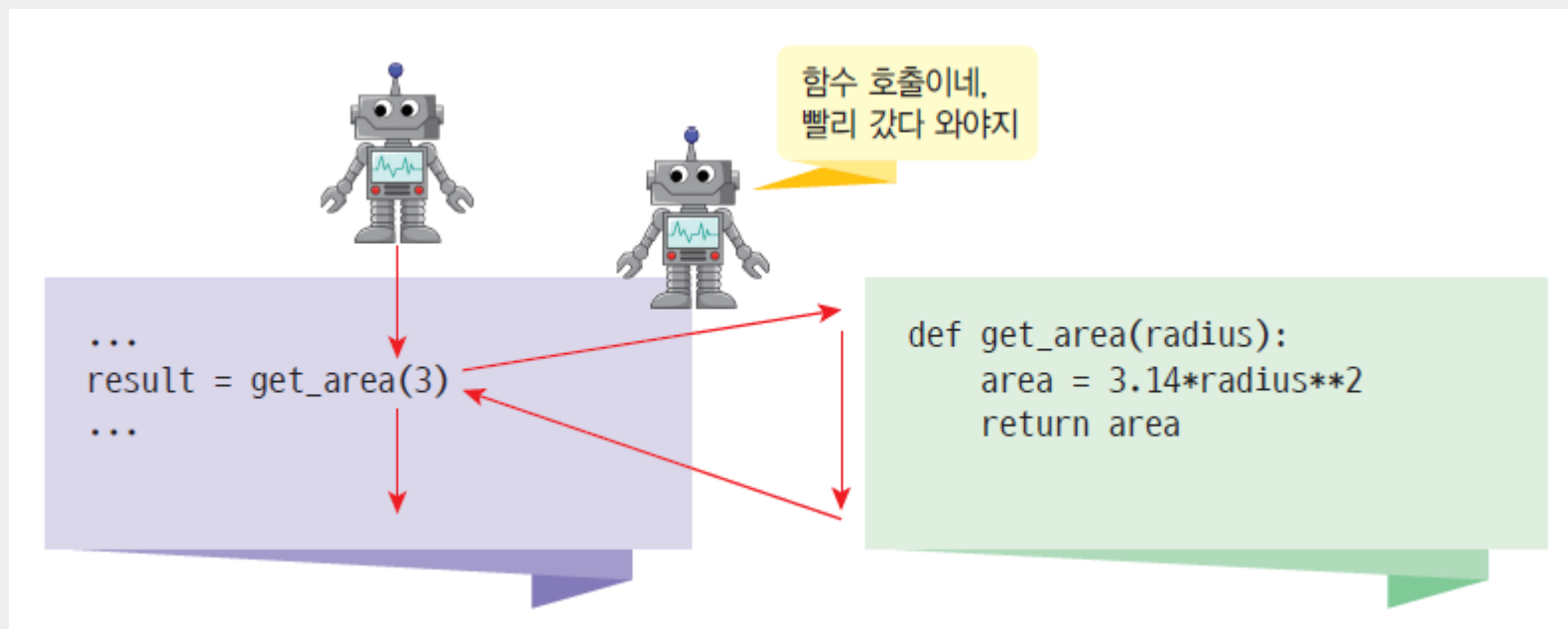
```
    실행문2
```

```
    .....
```

```
    return 반환값
```

함수 호출

- 함수 호출(function call)이란 `get_area()`과 같이 함수의 이름을 써주는 것이다. 함수가 호출되면 함수 안에 있는 문장들이 실행되며 실행이 끝나면 호출한 위치로 되돌아간다.



함수의 정의와 호출

매개변수와 반환값이 없는 경우

```
def hong_address():  
    print("서울특별시 종로구 1번지")  
    print("파이썬 빌딩 7층")  
    print("홍길동")
```

```
hong_address() #호출
```

```
서울특별시 종로구 1번지  
파이썬 빌딩 7층  
홍길동
```

- 한 번만 함수를 정의하면 언제든지 필요할 때마다 함수를 불러서 실행.

```
def hong_address():  
    print("서울특별시 종로구 1번지")  
    print("파이썬 빌딩 7층")  
    print("홍길동")
```

```
hong_address()  
hong_address()  
hong_address()
```

```
서울특별시 종로구 1번지  
파이썬 빌딩 7층  
홍길동  
서울특별시 종로구 1번지  
파이썬 빌딩 7층  
홍길동  
서울특별시 종로구 1번지  
파이썬 빌딩 7층  
홍길동
```

함수는 몇 번이고 호출될 수 있다.

- 함수는 일단 작성되면 몇 번이라도 호출이 가능하다.



```
...  
x = get_area(3)  
...  
y = get_area(20)
```

```
def get_area(radius):  
    area = 3.14*radius**2  
    return area
```



함수 내에서 또다른 함수를 호출 할 수 있다.

```
def hong():  
    print("홍길동")  
  
def greet():  
    print("안녕하세요 ")  
    hong()  
  
greet()
```

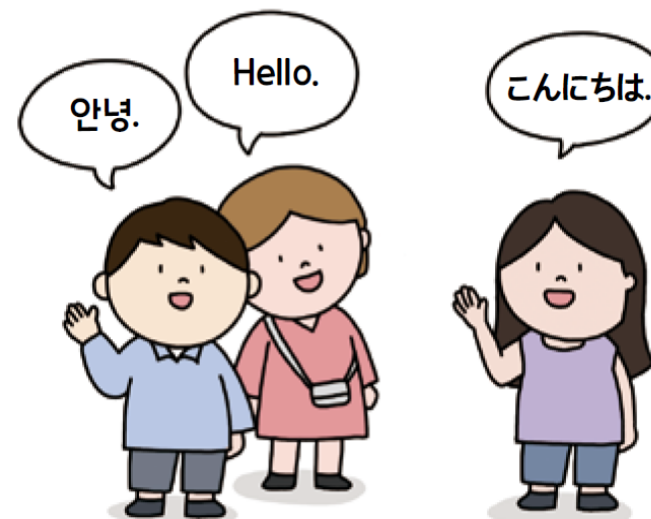
안녕하세요
홍길동

실습 : 국가에 따라 다른 인사

문제

출신 국가를 선택하면 해당하는 국가의 인사말이 출력되는 프로그램을 함수를 이용해서 만들어봅시다.

국가 선택	1. 한국	2. USA	3. Japan
인사말	안녕.	Hello.	こんにちは.



where are you from? 1.한국, 2.USA, 3.Japan 1
안녕.

where are you from? 1.한국, 2.USA, 3.Japan 2
Hello.

```
def introKor():  
    print('안녕!')
```

```
def introEng():  
    print('Hello!')
```

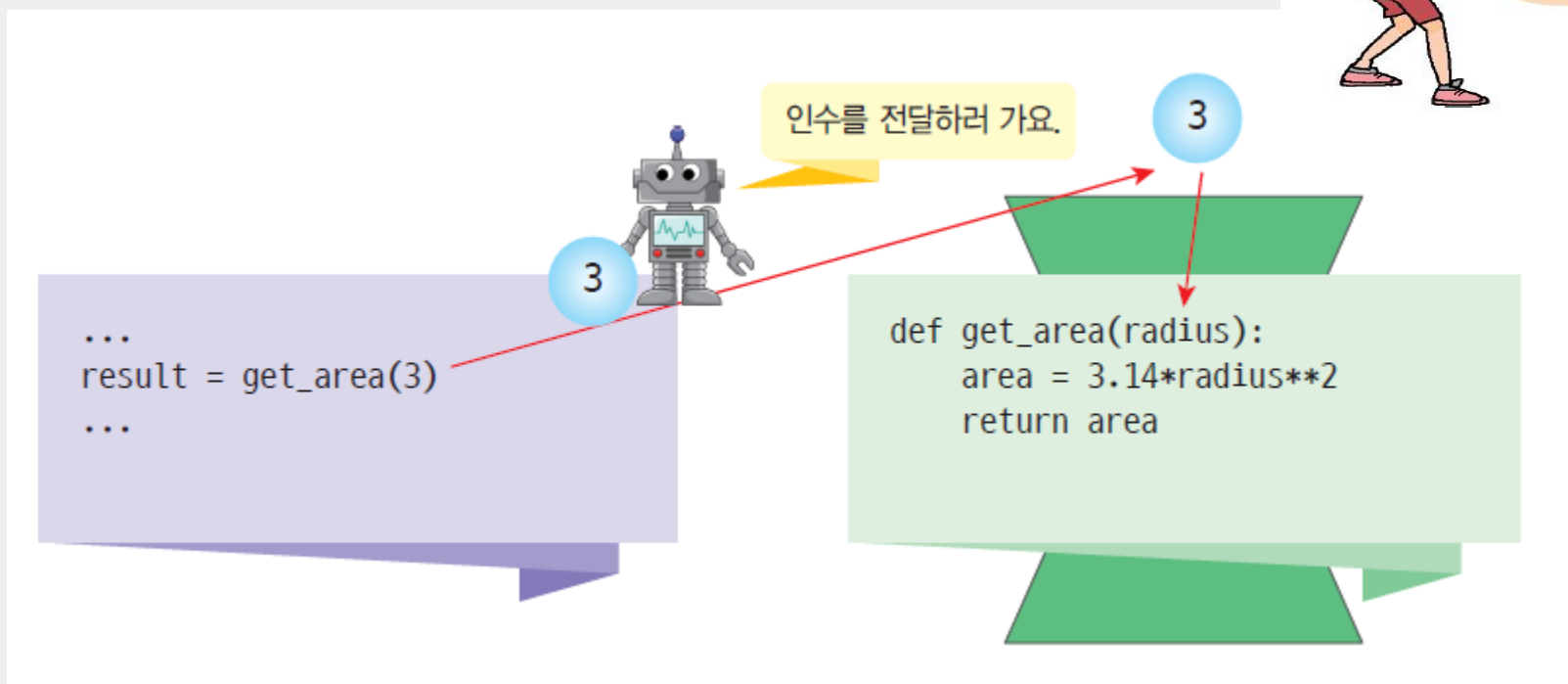
```
def introJap():  
    print('こんにちは!')
```

```
selectNum = int(input('where are you from? 1.한국, 2.USA, 3.Japan '))
```

```
if(selectNum == 1):  
    introKor()  
elif(selectNum == 2):  
    introEng()  
elif(selectNum == 3):  
    introJap()  
else:  
    introEng()
```

인수

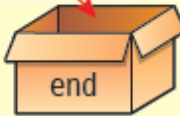
- 인수(argument) : 함수를 호출할 때 넘기는 값
- 매개변수(parameter) : 호출된 함수에서 사용할 값



매개변수 전달

매개변수(parameter)

```
value = get_sum( 1 , 10 )
```

```
def get_sum(  ,  ):  
    sum = 0  
    for i in range(start, end+1):  
        sum += i  
    return sum
```

인수(argument)


```
def address(name):  
    print("서울특별시 종로구 1번지")  
    print("파이썬 빌딩 7층")  
    print(name)
```

```
address("홍길동")
```

```
def address(name):
```

```
    print("서울특별시 종로구 1번지")
```

```
    print("파이썬 빌딩 7층")
```

```
    print(name)
```

```
a = input("이름을 입력하시오")
```

```
address(a)
```

실습

누구의 생일인가요? 홍길동

Happy Birthday to you!

Happy Birthday to you!

Happy Birthday dear 홍길동

Happy Birthday to you!

```
def happyBirthday(person):
```

```
    print("Happy Birthday to you!")
```

```
    print("Happy Birthday to you!")
```

```
    print("Happy Birthday, dear", person)
```

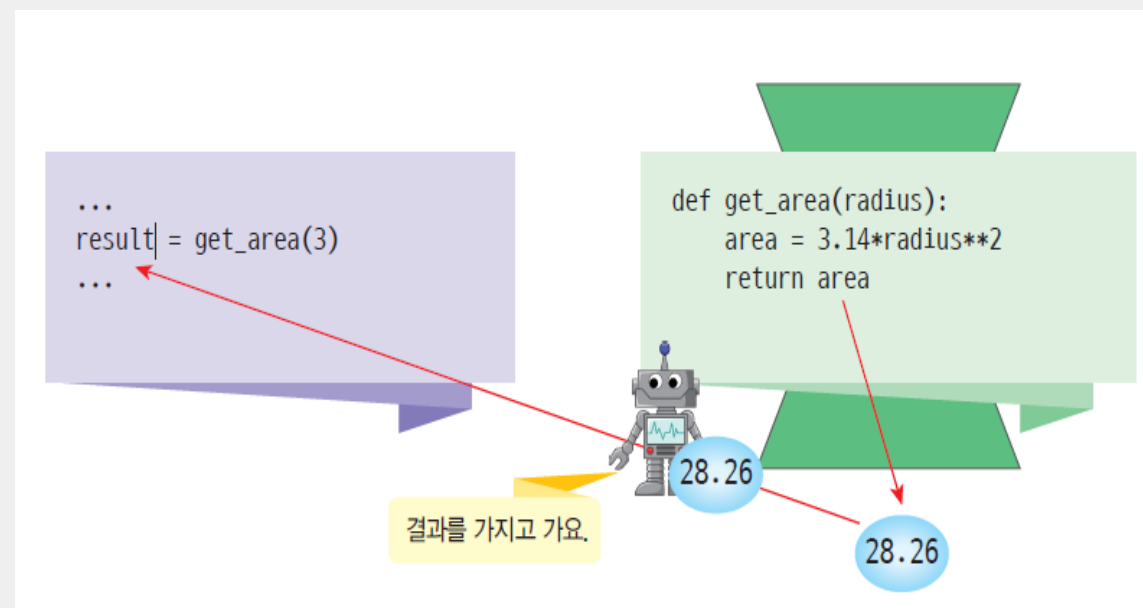
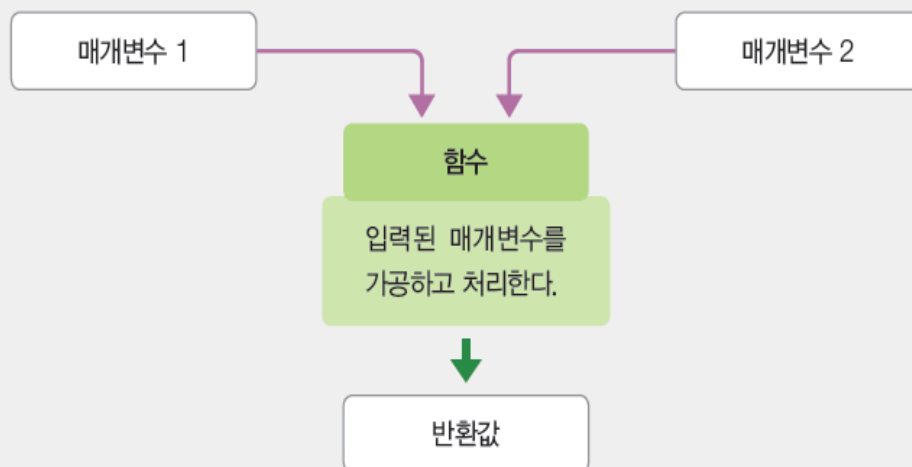
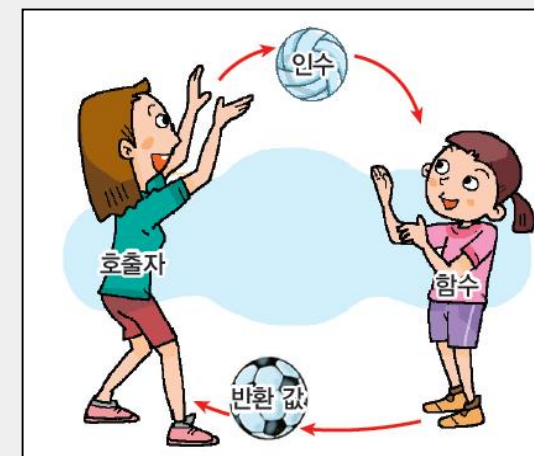
```
    print("Happy Birthday to you!")
```

```
name=input('누구의 생일인가요?')
```

```
happyBirthday(name)
```

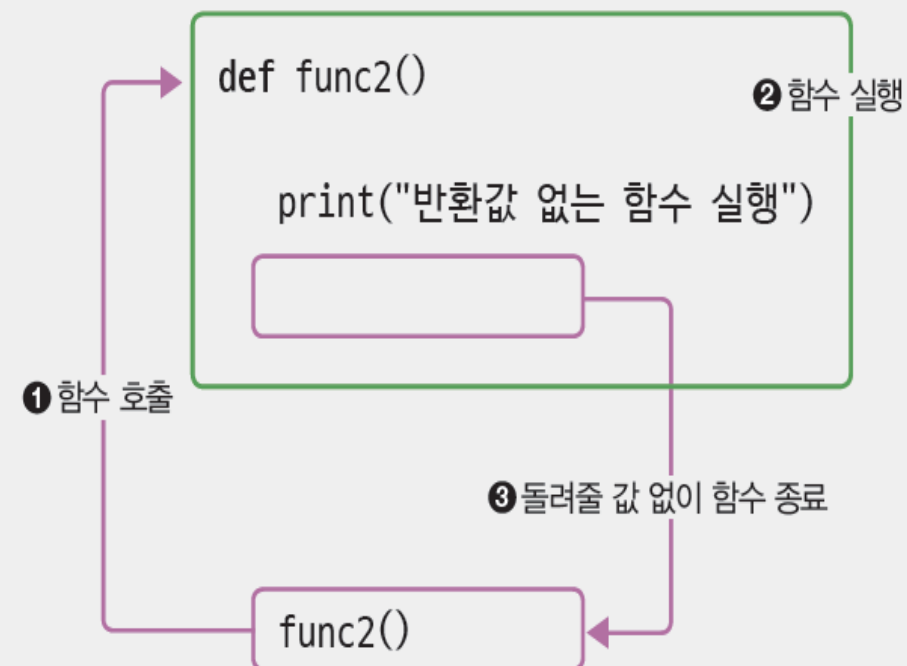
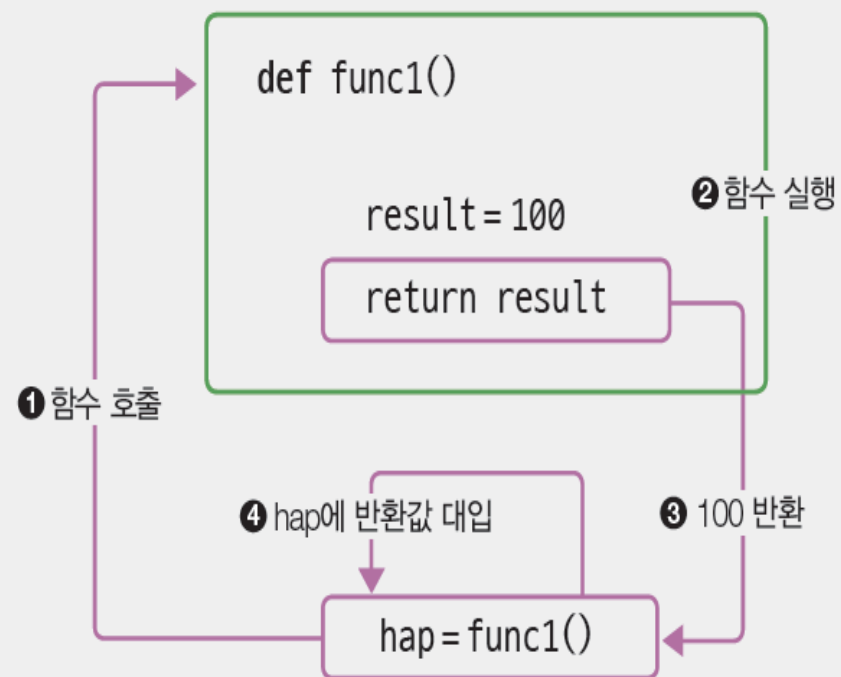
값 반환하기

- 함수는 값을 반환할 수 있다.
- 함수가 최종 결과로 내놓는 값을 반환 **return** 이라 함
- 함수를 호출한 곳으로 값을 돌려준다



반환값

- 반환값이 없는 함수
 - return문 생략. 또는 반환값 없이 return만 씀.



반환값이 없는 함수

```
def address(name):
```

```
    print("서울특별시 종로구 1번지")
```

```
    print("파이썬 빌딩 7층")
```

```
    print(name)
```

```
    return
```

```
a = input("이름을 입력하시오")
```

```
address(a)
```

```
def happyBirthday(person):
```

```
    print("Happy Birthday to you!")
```

```
    print("Happy Birthday to you!")
```

```
    print("Happy Birthday, dear", person)
```

```
    print("Happy Birthday to you!")
```

```
    return
```

```
name=input("누구의 생일인가요?")
```

```
happyBirthday(name)
```

서로 다른 인수로 호출될 수 있다.

```
def get_area(radius):  
    area = 3.14*radius**2  
    return area  
  
result1 = get_area(3)  
result2 = get_area(20)  
  
print("반지름이 3인 원의 면적=", result1)  
print("반지름이 20인 원의 면적=", result2)
```

```
반지름이 3인 원의 면적= 28.26  
반지름이 20인 원의 면적= 1256.0
```

여러 개의 값 반환하기

- 함수가 여러 개의 값을 반환할 수 있다.

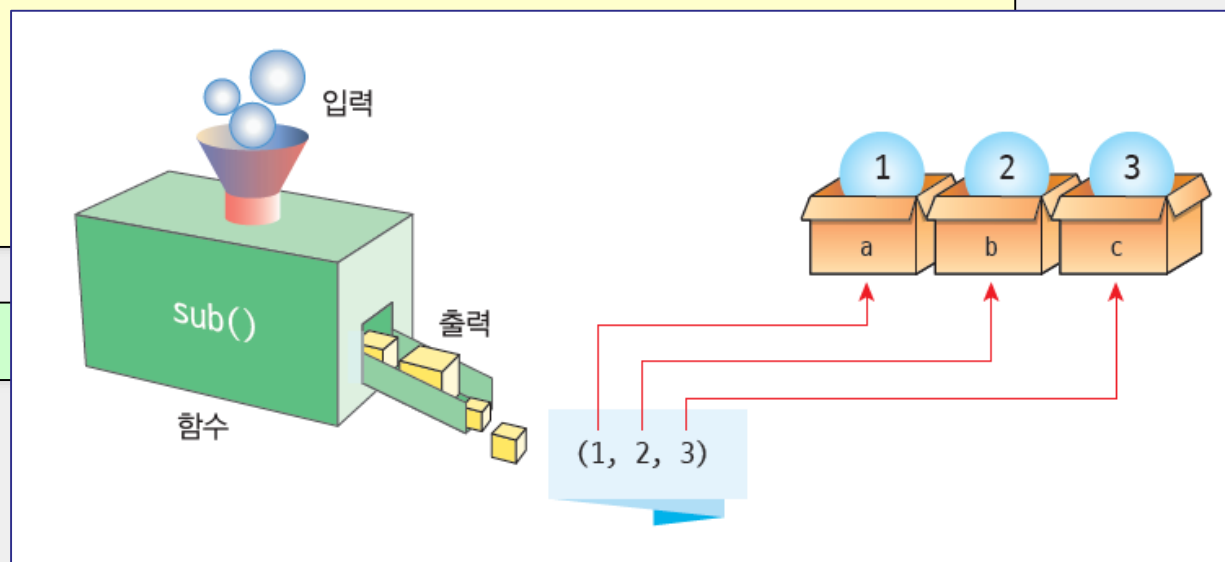
```
def get_input():  
    return 2, 3
```

```
x, y = get_input()           # x는 2이고 y는 3이다.
```

```
def sub():  
    return 1, 2, 3
```

```
a, b, c = sub()  
print(a, b, c)
```

1 2 3



실습: 여러 개의 값 반환

- 사용자로부터 이름과 나이를 입력받아서 동시에 반환하는 함수를 작성해보자.

이름:홍길동

나이:20

이름은 홍길동 이고 나이는 20 살입니다




```
def get_info():  
    name = input("이름:")  
    age = int(input("나이:"))  
    return name, age                # 2개의 값을 반환한다.  
  
st_name, st_age = get_info()        # 반환된 값을 풀어서 변수에 저장한다.  
print("이름은 ", st_name, "이고 나이는 ", st_age, "살입니다.")
```



실습 1/3

- ✓ 원의 반지름을 입력받아서 원의 면적을 계산하시오.
- ✓ 단. 면적 계산하는 함수를 따로 만들어서 사용.

실행결과

반지름을 입력하시오 : 10

반지름이 10인 원의 면적은 314.1592입니다.

```
def calculate_area (radius):
```

```
    area = 3.141592 * radius**2
```

```
    return area
```

```
r=float(input("반지름을 입력하시오"))
```

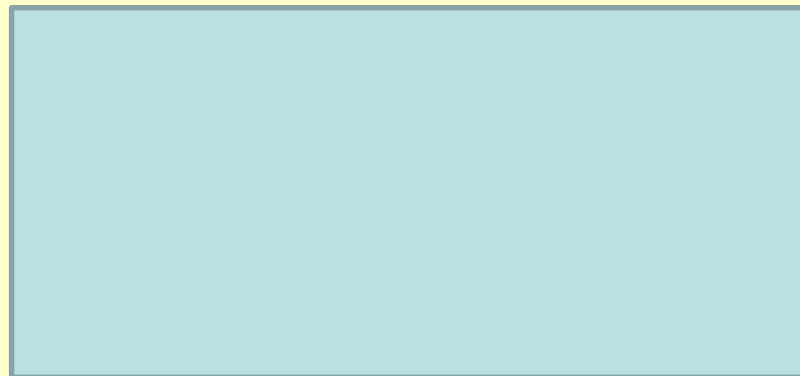
```
c_area = calculate_area(r)
```

```
print("반지름이 %f 인 원의 면적은 %f입니다" %(r, c_area))
```

실습 2/3

- 1부터 100까지 합을 구하는 함수 `cal_sum(100)`

```
def cal_sum(n):
```



```
hap = cal_sum(100)
```

```
print(hap)
```

실습 3/3

- 1부터 n까지 합을 구하는 함수

몇까지 합을 구할까요? 100
5050

```
def cal_sum(n):  
    s=0  
    for i in range(1,n+1):  
        s = s+i  
    return s
```

```
hap=cal_sum(100)  
print(hap)
```

```
def cal_sum(n):  
    s=0  
    for i in  
        s = s+i  
    return
```

```
num=int(input(  
hap=cal_sum(n  
print(hap)
```

```
def cal_sum(n):  
    s=0  
    for i in range(1,n+1):  
        s = s+i  
    return s
```

```
num=int(input("몇까지 합을 구할까요?"))  
print(cal_sum(num))
```

함수의 몸체는 나중에 작성할 수 있다.

- 함수의 헤더만 결정하고 몸체는 나중에 작성하고 싶은 경우 pass 키워드 사용.

```
def sub():  
    pass
```

함수의 순서

- 파이썬은 인터프리터 언어이기 때문에 함수의 순서가 중요하다.

```
result = get_area(3)
print("반지름이 3인 원의 면적=", result)

def get_area(radius):
    area = 3.14*radius**2
    return area
```

정의되지 않은 함수를 사용하였으므로 오류!!

함수의 순서

- 그러나 함수 내에서는 아직 정의되지 않은 함수를 호출할 수는 있다.

```
def main() :  
    result1 = get_area(3)  
    print("반지름이 3인 원의 면적=", result1)
```

main()

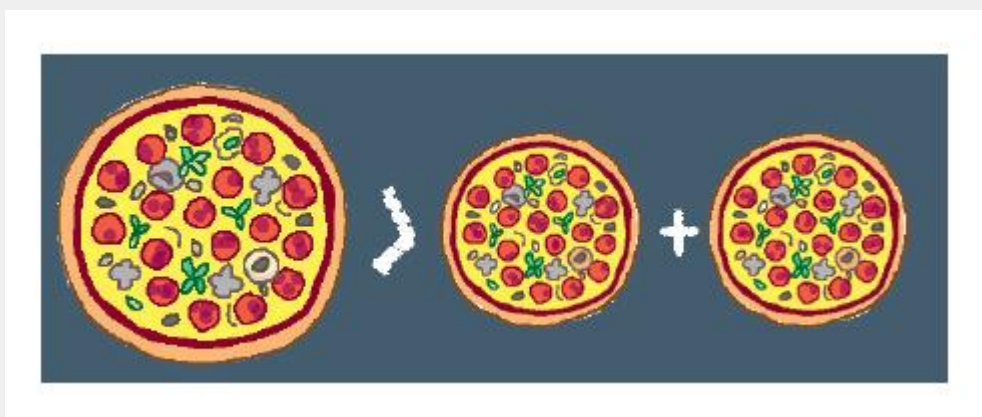
```
def get_area(radius):  
    area = 3.14*radius**2  
    return area
```

main()

함수 안에서는 정의되지 않은
다른 함수를 호출하여도 된다.

실습: 피자 크기 비교

- 원의 반지름을 받아서 피자의 면적을 계산하는 함수를 작성해서 사용해보자.



20cm 피자 2개의 면적: 2512.0

30cm 피자 1개의 면적: 2826.0

코딩:

```
def get_area(radius) :  
    if radius > 0 :  
        area = 3.14*radius**2  
    else :  
        area = 0  
    return area  
  
print("20cm 피자 2개의 면적:", get_area(20)+get_area(20))  
print("30cm 피자 1개의 면적:", get_area(30))
```



매개변수 전달방법

1. 매개변수의 개수를 정확하게 지정해서 전달
2. 기본값을 설정해놓고 생략할 수 있는 방법
3. 매개변수의 개수를 정해놓지 않고 가변적인 개수를 전달하는 방법

1. 매개변수의

- 개수가 일치하지 않

```
1  ## 함수 정의 부분
2  def para2_func(v1, v2) :
3      result=0
4      result=v1+v2
5      return result
6
7  def para3_func(v1, v2, v3) :
8      result=0
9      result=v1+v2+v3
10     return result
11
12  ## 변수 선언 부분
13  hap=0
14
15  ## 메인 코드 부분
16  hap=para2_func(10, 20)
17  print("매개변수 2개 함수 호출 결과==>%d" % hap)
18  hap=para3_func(10, 20, 30)
19  print("매개변수 3개 함수 호출 결과==>%d" % hap)
```

출력 결과

```
매개변수 2개 함수 호출 결과==> 30
매개변수 3개 함수 호출 결과==> 60
```

2. 기본값을 설정해 놓는 방법: 디폴트 인수

- 파이썬에서는 함수의 매개 변수가 기본값을 가질 수 있다. 이것을 디폴트 인수(default argument)라고 한다.

```
def greet(name, msg="별일없죠?"):
    print("안녕 ", name, msg)
```

```
greet("영희")
```

```
greet("철수", "잘 있었니? ")
```

range(start=0, stop, step=1)

시작값이다.

종료값이지만 stop은
포함되지 않는다.

한 번에 증가되는 값이다.

```
안녕 영희 별일없죠?
안녕 철수 잘 있었니?
```

2. 기본값을 설정해 놓는 방법: 디폴트 인수

```
1  ## 함수 정의 부분
2  def para_func(v1, v2, v3=0) :
3      result=0
4      result=v1+v2+v3
5      return result
6
7  ## 변수 선언 부분
8  hap=0
9
10 ## 메인 코드 부분
11 hap=para_func(10, 20)
12 print("매개변수 2개 함수 호출 결과==>%d" % hap)
13 hap=para_func(10, 20, 30)
14 print("매개변수 3개 함수 호출 결과==>%d" % hap)
```

키워드 인수

- 키워드 인수(keyword argument)는 인수의 이름을 명시적으로 지정해서 값을 매개 변수로 전달하는 방법이다.

```
def sub(x, y, z):  
    print("x=", x, "y=", y, "z=", z)
```

```
>>> sub(10, 20, 30) 위치인수(positional argument)
```

```
x= 10 y= 20 z= 30
```

```
>>> sub(x=10, y=20, z=30)
```

```
x= 10 y= 20 z= 30
```

```
>>> sub(10, y=20, z=30)
```

```
x= 10 y= 20 z= 30
```

```
>>> sub(x=10, 20, 30)
```

```
sub(x=10, 20, 30)
```

```
^
```

```
SyntaxError: positional argument follows keyword argument
```

키워드인수 : 인수들이 어떤 순서로 전달되어도 상관없음
= sub(y=20, x=10, z=30)

키워드인수 뒤에 위치인수가 나올 수 없음

3. 가변 인수

- 인수의 개수를 모르는 경우- * 사용
- 함수의 입력파라미터의 갯수를 미리 알 수 없거나, 0부터 n개의 파라미터를 받아들이도록
- 인수가 튜플형태로 전달

```
def varfunc(*args):  
    print (args)  
  
print("하나의 값으로 호출")  
varfunc(10)  
  
print("여러 개의 값으로 호출")  
varfunc(10, 20, 30)
```

```
하나의 값으로 호출  
(10,)  
여러 개의 값으로 호출  
(10, 20, 30)
```


예제



```
def add(*numbers) :  
    sum = 0  
    for n in numbers:  
        sum = sum + n  
    return sum  
  
print(add(10, 20))  
print(add(10, 20, 30))
```

```
30  
60
```



실습

- 시험성적을 출력하는 프로그램으로 과목수가 3과목에서 4과목으로, 또는 5과목으로 변경되어도 정상적으로 실행되는 프로그램

총점 : 240점

평점 : 80.0점

총점 : 365점

평점 : 91.25점

총점 : 455점

평점 : 91.0점

총점 : 240점

평균 : 80.0점

총점 : 365점

평균 : 91.25점

총점 : 455점

평균 : 91.0점

```
def AverageScore(*scores):
```

```
    sum = 0
```

```
    cnt = len(scores)
```

```
    for i in scores:
```

```
        sum = sum + i
```

```
    print('총점: ', sum, '점')
```

```
    print('평균: ', sum / cnt, '점')
```

```
    print('-----')
```

```
AverageScore(80, 90, 70)
```

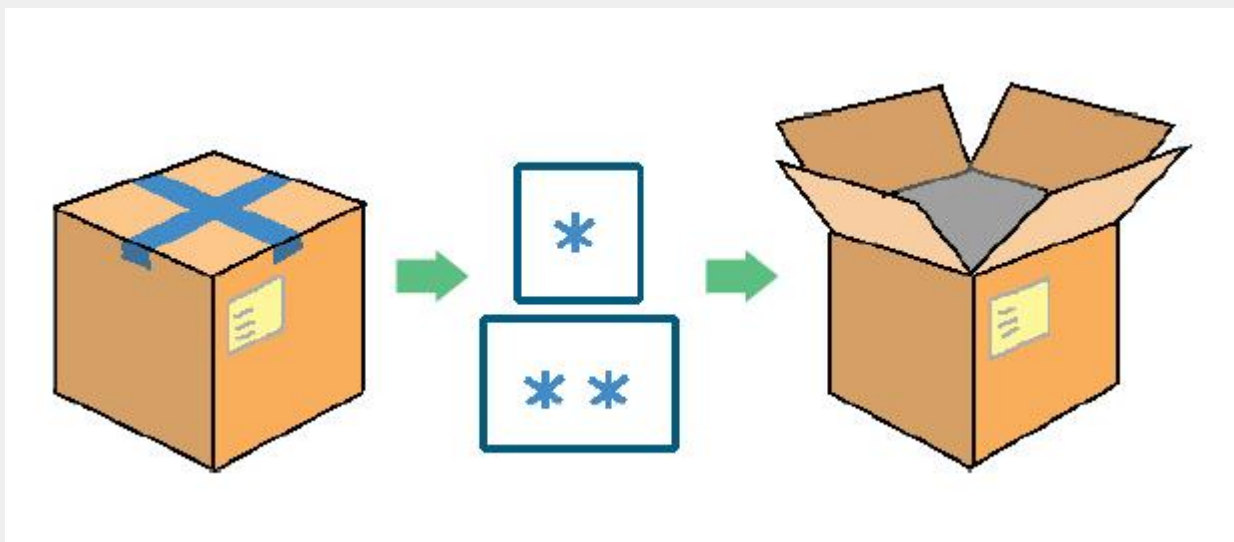
```
AverageScore(90, 85, 90, 100)
```

```
AverageScore(95, 80, 100, 95, 85)
```

* 연산자로 언패킹하기

리스트, 튜플, 딕셔너리, 세트, 문자열

- 단일 별표 연산자 *는 파이썬이 제공하는 모든 반복 가능한 객체(iterable)를 언패킹할 수 있고 이중 별표 연산자 **는 딕셔너리 객체를 언패킹할 수 있다.



예제



```
>>> alist = [ 1 , 2 , 3 ]
```

```
>>> print(alist)
```

```
[1, 2, 3]
```

```
>>> alist = [ 1 , 2 , 3 ]
```

```
>>> print(*alist)
```

```
1 2 3
```

예제



```
def sum(a, b, c):  
    print(a + b + c)  
  
alist = [1, 2, 3]  
print(sum(*alist))    #sum
```

6

```
def sum(a):  
    sum=0  
    for i in a:  
        sum=sum+i  
    return sum
```

```
alist = [1, 2, 3]  
print(sum(alist))
```

실습 1/2: 환영 문자열 출력 함수

- 전광판에 “환영합니다” 문자열을 여러 번 출력하는 함수 `display(msg, count)`를 작성해보자. Count를 지정하지 않은 경우 1회만 출력하도록.

`display("환영합니다.", 5)`

환영합니다.
환영합니다.
환영합니다.
환영합니다.
환영합니다.



```
def display(msg, count=1) :  
    for k in range(count) :  
        print(msg)
```

```
display("환영합니다.", 5)
```


실습 2/2 : 주급 계산 프로그램

- 주단위로 봉급을 받는 알바생이 있다고 하자. 현재 시급과 일한 시간을 입력하면 주급을 계산해주는 함수 `weeklyPay(rate, hour)`를 만들고 이 함수를 호출하여 주급을 출력하는 프로그램을 작성해보자. 30시간이 넘는 근무 시간에 대해서는 시급의 1.5 배를 지급한다고 하자.

시급을 입력하시오:10000

근무 시간을 입력하시오:38

주급은 420000.0

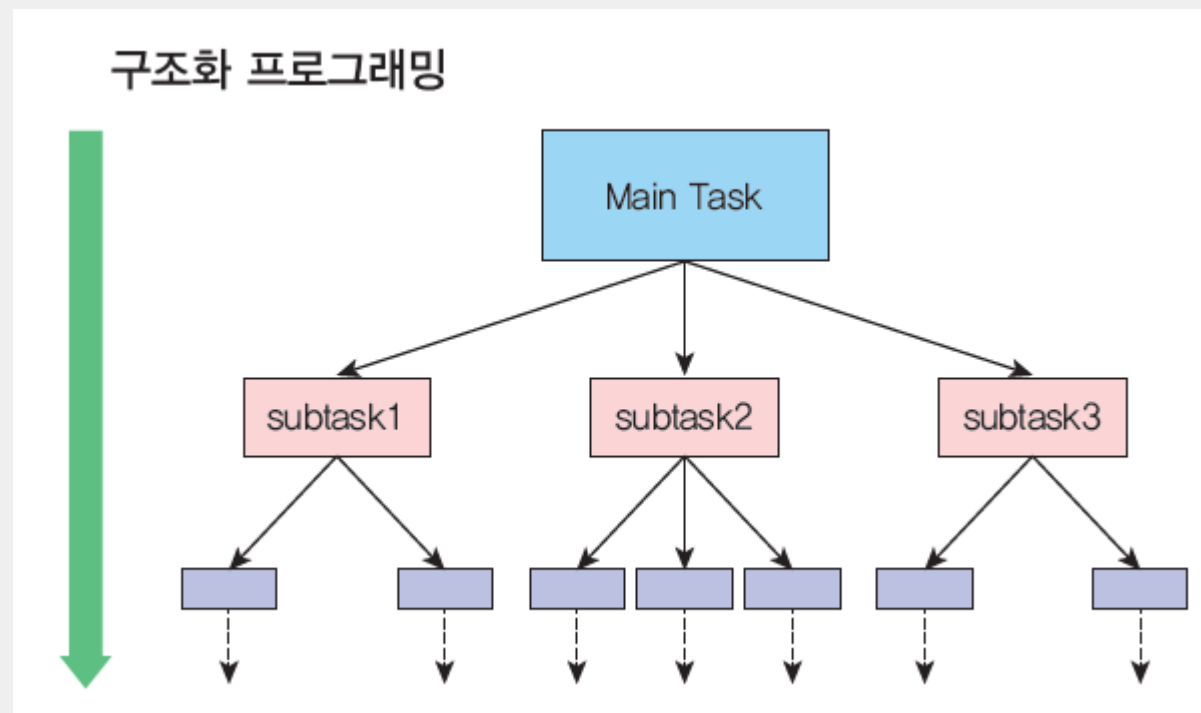


```
def weeklyPay( rate, hour ):  
    if hour > 30:  
        money = rate*30 + 1.5*rate*(hour-30)  
    else:  
        money = rate*hour  
    return money
```

```
rate = int(input("시급을 입력하시오:"))  
hour = int(input("근무 시간을 입력하시오:"))  
print("주급은 " ,weeklyPay(rate, hour))
```

함수를 사용하는 이유

- 소스 코드의 중복성을 없애준다.
- 한번 제작된 함수는 다른 프로그램을 제작할 때도 사용이 가능하다.
- 복잡한 문제를 단순한 부분으로 분해할 수 있다.



실습: 구조화 프로그래밍 실습

- 온도를 변환해주는 프로그램을 작성해보자.

1. 섭씨 온도 → 화씨 온도
2. 화씨 온도 → 섭씨 온도
3. 종료

메뉴를 선택하세요: 1

섭씨 온도를 입력하시오: 37.0

화씨 온도 = 98.6

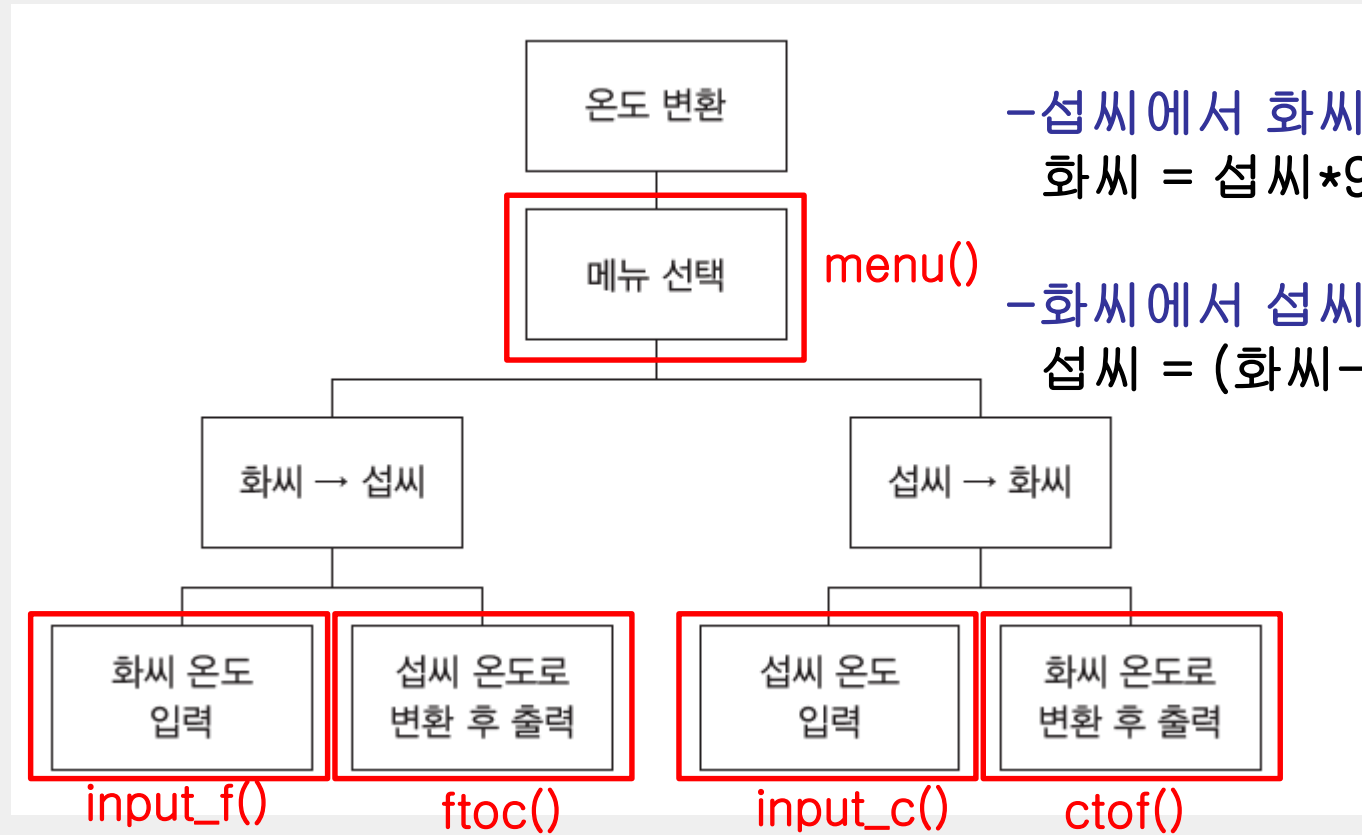
1. 섭씨 온도→화씨 온도
2. 화씨 온도→섭씨 온도
3. 종료

메뉴를 선택하세요: 3

구조도



인하대학교



-섭씨에서 화씨 변환 공식

$$\text{화씨} = \text{섭씨} * 9.0 / 5.0 + 32$$

menu()

-화씨에서 섭씨 변환 공식

$$\text{섭씨} = (\text{화씨} - 32.0) * 5.0 / 9.0$$



```
def menu() :  
    print("1. 섭씨 온도->화씨 온도")  
    print("2. 화씨 온도->섭씨 온도")  
    print("3. 종료")  
    selection = int(input("메뉴를 선택하세요: "))  
    return selection
```

```
def ctof(c) :  
    temp = c*9.0/5.0 + 32  
    return temp
```

```
def ftoc(f) :  
    temp = (f-32.0)*5.0/9.0  
    return temp
```

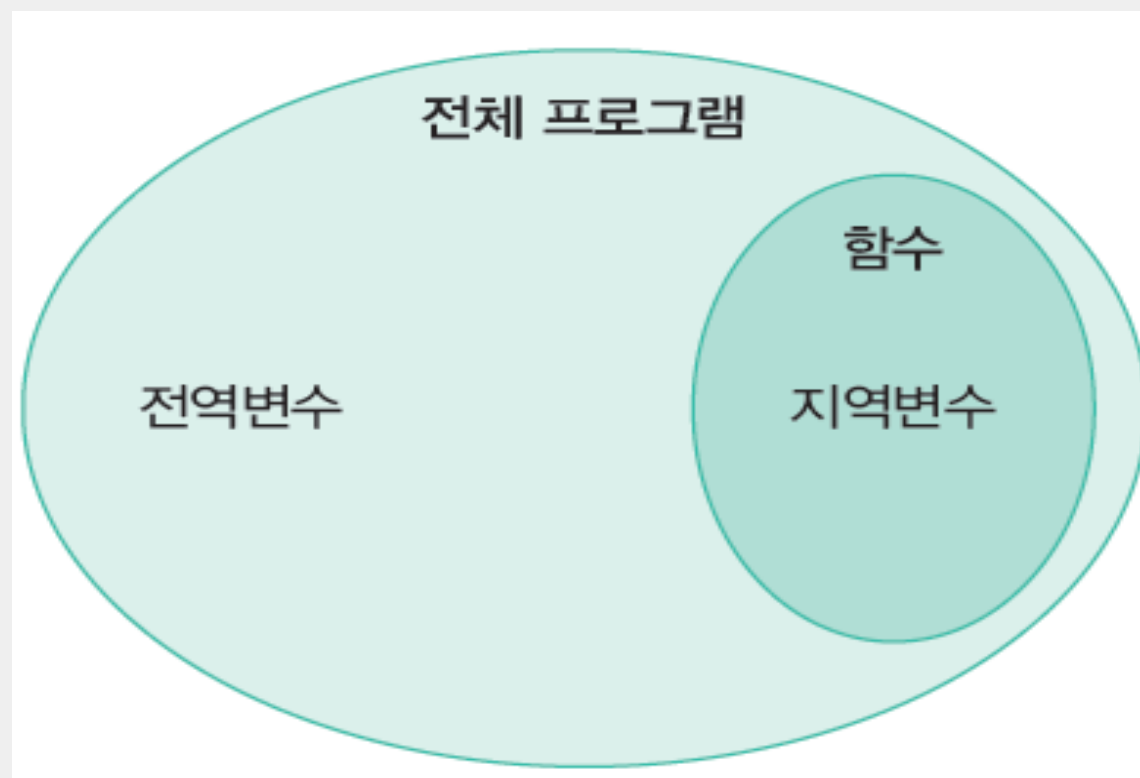
```
def input_f() :  
    f = float(input("화씨 온도를 입력하세요: "))  
    return f
```

```
def input_c() :  
    c = float(input("섭씨 온도를 입력하세요: "))  
    return c
```

```
while True:  
    index = menu()  
    if index == 1 :  
        t = input_c()  
        t2 = ctof(t)  
        print("화씨 온도 = ", t2, "\n")  
    elif index == 2 :  
        t = input_f()  
        t2 = ftoc(t)  
        print("섭씨 온도 = ", t2, "\n")  
    else :  
        break
```

변수의 범위

- 지역 변수: 함수 안에서 선언되는 변수
- 전역 변수: 함수 외부에서 선언되는 변수



변수의 범위

- 지역변수는 한정된 지역(Local)에서만 사용되는 변수
- 전역변수는 프로그램 전체(Global)에서 사용되는 변수

① 지역변수의 생존 범위

함수 1

a = 10

a가 뭔지 함수 1에서 안다.

함수 2

a가 뭔지 함수 2에서 모른다.

② 전역변수의 생존 범위

함수 1

b가 뭔지 함수 1에서 안다.

함수 2

b가 뭔지 함수 2에서 안다.

b = 20

- ① 에서 a는 현재 함수 1 안에 선언. 즉 a는 함수 1 안에서만 사용될 수 있음
- ② b는 함수(함수 1, 함수 2) 안이 아니라 바깥에 선언, 모든 함수에서 b의 존재를 안다.

지역변수

- 지역 변수는 함수 안에서만 사용가능

```
def calculate_area (radius):  
    result = 3.14 * radius**2  
    return result  
  
r = float(input("원의 반지름: "))  
area = calculate_area(r)  
print(result)
```

원의 반지름: 5.0

Traceback (most recent call last):

File "D:\s.py", line 7, in <module>

print(result)

NameError: name 'result' is not defined

전역변수

- 전역 변수는 어디서나 사용가능

```
def calculate_area ():  
    result = 3.14 * r**2  
    return result
```

전역변수 r을 사용한다.

```
r = float(input("원의 반지름: "))  
area = calculate_area()  
print(area)
```



```
원의 반지름: 5.0  
78.5
```

전역 변수



인하대학교

```
gx = 100
```

```
def myfunc():  
    print(gx)
```

```
myfunc()  
print(gx)
```

```
def happyBirthday( ):
```

```
    print("Happy Birthday to you!")
```

```
    print("Happy Birthday to you!")
```

```
    print("Happy Birthday, dear", name)
```

```
    print("Happy Birthday to you!")
```

```
name=input("누구의 생일인가요?")
```

```
happyBirthday( )
```

```
100
```

```
100
```

예제

```
1  ## 함수 정의 부분
2  def func1() :
3      a=10    # 지역변수
4      print("func1()에서 a의 값 %d" % a)
5
6  def func2() :
7      print("func2()에서 a의 값 %d" % a)
8
9  ## 변수 선언 부분
10 a=20    # 전역변수
11
12 ## 메인 코드 부분
13 func1()
14 func2()
```

출력 결과

```
func1()에서 a의 값 10
func2()에서 a의 값 20
```



지역 변수는 함수마다 동일한 이름을 사용할 수 있다.

```
def myfunc() :
```

```
    x = 200
```

```
    print(x)
```

```
def main() :
```

```
    x = 100
```

```
    print(x)
```

```
myfunc()
```

```
main()
```

200

100

함수 안에서 전역 변수 변경하기

```
gx = 100
```

```
def myfunc():  
    gx = 200  
    print(gx)
```

```
myfunc()  
print(gx)
```

```
200  
100
```

변경되지 않는다! → 함수 안에서 변수에 값을 저장하면 새로운 지역 변수가 생성된다.

```
def calculate_area (radius):  
    area = 3.14 * radius**2 # 전역변수 area에 계산 값을 저장하려고 했다!  
    return
```

```
area = 0  
r = float(input("원의 반지름: "))  
calculate_area(r)  
print(area)
```

원의 반지름: 5.0

0

함수 안에서 전역 변수 변경하기

```
gx = 100
```

```
def myfunc():
```

```
    global gx
```

```
    gx = 200
```

```
    print(gx)
```

전역 변수 gx를 사용한다.

```
myfunc()
```

```
print(gx)
```

```
200
```

```
200
```